

Artificial Intelligence and RAG Pipeline

1. What is Artificial Intelligence?

Artificial Intelligence (AI) is a field of computer science focused on building systems that can perform tasks that normally require human intelligence. These tasks include understanding language, recognizing images, making predictions, solving problems, and generating content.

Modern AI systems often use machine learning models trained on large datasets. Deep learning, a branch of machine learning, uses neural networks with many layers to learn complex patterns from data.

2. What is Generative AI?

Generative AI refers to AI systems that can create new content based on patterns learned during training. Generative AI models can produce text, images, audio, video, and computer code.

Large language models (LLMs) are a major category of generative AI. They process text and generate responses based on the input provided to them.

Examples of tasks performed by LLMs include question answering, summarization, translation, classification, and code generation.

3. What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation, commonly called RAG, is an architecture that combines information retrieval with a generative language model.

Instead of asking an LLM to answer a question only from information learned during training, a RAG system first retrieves relevant information from an external knowledge base. The retrieved information is then provided to the LLM as context.

A typical RAG pipeline contains five major stages:

1. Document loading
2. Document splitting
3. Text embedding
4. Vector storage and retrieval
5. Answer generation

4. Document Loading

The first stage of a RAG pipeline is document loading.

Documents can come from many sources, including PDF files, Word documents, text files, web pages, databases, and spreadsheets.

A document loader reads the source file and converts its contents into a format that the RAG application can process.

For example, a PDF containing information about artificial intelligence can be loaded and converted into text documents.

5. Document Splitting

Large documents are usually divided into smaller pieces called chunks.

Chunking is important because language models have limits on how much information they can process in a single request. Smaller chunks also make retrieval more precise.

A common approach is to use a chunk size of approximately 500 to 1,000 characters with some overlap between neighboring chunks.

For example, a 10-page AI document might be divided into dozens of smaller chunks.

6. Embeddings

An embedding model converts text into numerical vectors.

Text with similar meanings generally produces vectors that are close together in the embedding space.

For example, the following questions are semantically similar:

"What is RAG?"

"Explain retrieval augmented generation."

"What does retrieval-augmented generation mean?"

An embedding model can represent these questions as vectors so that a vector database can search for semantically related documents.

7. Vector Databases

A vector database stores embeddings and allows applications to search for similar vectors.

Qdrant is a vector database that can be used in RAG applications.

In a RAG pipeline, document chunks are converted into embeddings and stored in a Qdrant collection. When a user asks a question, the question is also converted into an embedding. Qdrant then searches for document chunks with similar vectors.

8. Retrieval

Retrieval is the process of finding relevant document chunks for a user's question.

Suppose a user asks:

"What are the main stages of a RAG pipeline?"

The retriever searches the vector database and may return chunks describing document loading, splitting, embeddings, vector storage, retrieval, and answer generation.

Only the most relevant chunks should normally be passed to the language model.

9. Document Grading

Some RAG systems use an LLM to evaluate whether retrieved documents are actually relevant to the user's question.

For example, if five chunks are retrieved, the grading process may classify them as:

Chunk 1: Relevant

Chunk 2: Relevant

Chunk 3: Not relevant

Chunk 4: Relevant

Chunk 5: Not relevant

The system can then send only the relevant chunks to the final answer-generation step.

Document grading can improve answer quality, but it requires additional LLM calls and therefore increases computational cost.

10. Ollama

Ollama allows language models to run locally on a computer.

A RAG application can use Ollama as its local language model provider instead of a cloud inference service.

For example, a RAG application may use an Ollama chat model to generate answers and an Ollama embedding model to create document embeddings.

Running models locally can reduce dependency on external API services and is useful for development and testing.

11. LangChain

LangChain is a framework used to build applications around language models.

It provides components for document loading, text splitting, embeddings, vector stores, retrievers, prompts, and LLM interactions.

A RAG application can combine these components into a complete question-answering system.

12. LangGraph

LangGraph can be used to build stateful workflows involving language models.

A RAG workflow can be represented as a graph containing nodes such as:

- Retrieve documents
- Grade documents
- Generate an answer
- Rewrite the question
- Search again

The graph controls how the application moves between these steps.

13. FastAPI

FastAPI is a Python framework for building APIs.

A RAG application can expose an endpoint such as:

POST /query

A client sends a question to the API, and the RAG pipeline retrieves relevant information and returns an answer.

For example:

Question:

"What is a vector database?"

Answer:

"A vector database stores numerical representations of data and supports similarity searches over those vectors."

14. Complete RAG Architecture

A complete local RAG application can use the following architecture:

User Question

→ FastAPI

→ LangGraph

- Retriever
- Qdrant
- Relevant Documents
- Ollama
- Generated Answer
- FastAPI Response

The ingestion pipeline is separate:

PDF Document

- Document Loader
- Text Splitter
- Ollama Embeddings
- Qdrant

15. Advantages of RAG

RAG provides several advantages.

First, it allows an application to use information that was not part of the language model's original training data.

Second, documents can be updated without retraining the language model.

Third, retrieved context can make answers more relevant to a specific organization's documents.

RAG is commonly used for document question answering, enterprise knowledge bases, research assistants, customer-support systems, and internal search applications.

16. Example Questions

The following questions can be used to test a RAG application using this document:

1. What is Artificial Intelligence?
2. What is Generative AI?
3. What does RAG stand for?
4. What are the major stages of a RAG pipeline?
5. Why are documents split into chunks?
6. What are embeddings?
7. What is Qdrant?
8. What happens during retrieval?
9. Why is document grading used?

10. What is Ollama?
11. What is LangChain?
12. What is LangGraph?
13. What role does FastAPI play in a RAG application?
14. What is the difference between ingestion and retrieval?
15. Describe the complete RAG architecture.

17. Key Takeaway

A RAG system combines a knowledge base with a language model. Documents are loaded, split into chunks, converted into embeddings, and stored in a vector database such as Qdrant. When a user asks a question, relevant chunks are retrieved and supplied to an LLM such as an Ollama model. The LLM then generates an answer based on the retrieved context.